This will find all of the `<span>` elements anywhere on the web page that have an attribute named `class` that's equal to `destination-title`.

You could make this more generic by using a wildcard (`*`) instead of `span`:

```
driver.findElements(By.xpath("//*[@class='destination-title']"));
```

This would find elements whose class was exactly equal to `destination-title`. Unfortunately, modern web applications will sometimes add extra classes to the `class` attribute, so you can't rely on an exact match. A more reliable solution would be to use the XPath `contains()` function, matching elements that have a `class` attribute with a value that contains `destination-title`:

```
driver.findElements(By.xpath("//*[contains(@class,'destination-title')]"));
```

The full power of XPath becomes more apparent when you need to find elements based on their content—something that's not currently supported in CSS. For example, the featured destinations you saw earlier are rendered in HTML like this:

```
<div class="featured-destination"...>
    <img src="img/singapore.png"></img>
    <span class="destination-title">Singapore</span>
    <span class="destination-price">$900</span>
</div>
<div class="featured-destination">...</div>
```

You could find the `<span>` element containing the text `Singapore` using the following XPath expression:

```
//span[.='Singapore']
```

You could take this further. Suppose you need to find the price displayed for the Singapore featured destination. XPath supports relative paths using the ".." notation, so you could find the neighboring `<span>` notation with a class of `destination-price` like this:

```
//span[.='Singapore']/../span[contains(@class,'destination-price')]
```

XPath isn't without its disadvantages. XPath expressions are generally more verbose and less readable than CSS selectors. XPath expressions can also be fragile if they aren't well-crafted. XPath has no native support in Internet Explorer, so tests that use XPath on Internet Explorer may run very slowly. But XPath is more powerful than CSS, and there are cases where XPath will be the only way to reliably identify the elements you're looking for.

Using nested lookups

When you write automated tests with WebDriver, it's important to keep expressions as simple and readable as possible. Simpler expressions tend to be easier to understand and to maintain, and in many cases they're more reliable. One useful strategy when it comes to writing simpler WebDriver code is to use nested lookups.

So far, you've found web elements using the WebDriver instance. But you can also call the `findElement()` and `findElements()` methods directly on `WebElement` instances.